LAPORAN TUGAS

Penyimpanan Lokal di Flutter: SQLite (sqflite) dan Hive

Praktikum Mobile Programming

Nama : Arief Fadhilah Rahman
NPM : 065124122
Mata Kuliah : Praktikum Mobile Programming
Aplikasi : Pengeluaran Harian (Flutter)

1. Pendahuluan

Penyimpanan data lokal dibutuhkan aplikasi mobile agar data tetap ada walaupun aplikasi ditutup atau perangkat tidak terhubung internet. Pada tugas ini dibuat aplikasi Pengeluaran Harian di Flutter untuk mencatat pengeluaran (keterangan, jumlah, kategori, dan tanggal). Penyimpanan utama memakai SQLite melalui package sqflite, dan satu teknologi local storage alternatif yang dieksplorasi adalah Hive.

Laporan ini disusun mengikuti lima poin tugas:

1. Mempelajari dasar penggunaan package sqflite di Flutter.
2. Membuat database dan skema tabel menggunakan SQLite.
3. Menerapkan fungsi operasi CRUD (Create, Read, Update, Delete) pada SQLite.
4. Mencari dan mengeksplorasi satu teknologi local storage alternatif di Flutter (Hive).
5. Membuat ringkasan (summary) serta dokumentasi hasil uji coba SQLite dan storage alternatif tersebut.

2. Dasar Penggunaan Package sqflite

sqflite adalah plugin Flutter untuk database SQLite di Android dan iOS. Seluruh data disimpan dalam satu file .db di perangkat. Langkah dasar penggunaannya:

1. Tambahkan dependency sqflite dan path pada pubspec.yaml, lalu jalankan flutter pub get.
2. Tentukan lokasi file database dengan getDatabasesPath() dan join() dari package path.
3. Buka database dengan openDatabase(path, version, onCreate). Tabel dibuat di dalam callback onCreate yang hanya dipanggil saat database pertama kali dibuat.
4. Akses data dengan method insert, query, update, dan delete (atau rawQuery dan execute untuk SQL langsung).
5. Semua operasi bersifat asynchronous sehingga dipanggil dengan async dan await.

Dependency yang dipakai pada pubspec.yaml:

```
dependencies:
  sqflite: ^2.4.2
  path: ^1.9.1
  hive_flutter: ^1.1.0
```

3. Database dan Skema Tabel

Database bernama pengeluaran.db (versi 1) dengan satu tabel bernama pengeluaran. Skema tabel:

Tabel 1. Skema tabel pengeluaran

Kolom	Tipe	Keterangan
id	INTEGER	Primary key, autoincrement
judul	TEXT	NOT NULL, keterangan pengeluaran
jumlah	INTEGER	NOT NULL, nominal dalam Rupiah
kategori	TEXT	NOT NULL (Makanan, Transportasi, Belanja, Hiburan, Lainnya)
tanggal	TEXT	NOT NULL, format ISO 8601

Skema dibuat pada callback onCreate di file lib/data/sqlite_repo.dart:

```
CREATE TABLE pengeluaran(
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  judul TEXT NOT NULL,
  jumlah INTEGER NOT NULL,
  kategori TEXT NOT NULL,
  tanggal TEXT NOT NULL
)
```

Saat database pertama kali dibuat, tujuh data contoh dimasukkan agar aplikasi tidak kosong ketika pertama dibuka.

4. Implementasi CRUD pada SQLite

Operasi CRUD diimplementasikan pada kelas SqliteRepo (lib/data/sqlite_repo.dart) yang mengikuti antarmuka PengeluaranRepo.

Tabel 2. Operasi CRUD pada SQLite

Operasi	Method	Fungsi sqflite
Create	tambah()	db.insert('pengeluaran', {...})
Read	semua()	db.query('pengeluaran', orderBy: 'tanggal DESC, id DESC')
Update	ubah()	db.update('pengeluaran', {...}, where: 'id = ?', whereArgs: [id])
Delete	hapus()	db.delete('pengeluaran', where: 'id = ?', whereArgs: [id])

Cara penggunaan pada aplikasi (tab SQLite):

- **Create:** tekan tombol Tambah, isi keterangan, jumlah, kategori, dan tanggal, lalu Simpan.
- **Read:** daftar pengeluaran tampil berurutan dari yang terbaru, lengkap dengan total dan rincian per kategori.
- **Update:** ketuk salah satu item, ubah datanya, lalu Simpan.
- **Delete:** geser item ke kiri. Tersedia tombol Urungkan jika salah hapus.

Form dilengkapi validasi: keterangan tidak boleh kosong dan jumlah harus lebih dari nol.

5. Eksplorasi Local Storage Alternatif: Hive

Teknologi alternatif yang dipilih adalah Hive, yaitu database NoSQL ringan berbasis key-value yang ditulis dengan Dart murni. Data disimpan dalam box, tanpa skema dan tanpa SQL. Hive dipilih karena penggunaannya sederhana dan sering dipakai untuk data lokal yang tidak kompleks seperti pengaturan, cache, atau catatan sederhana.

Langkah dasar penggunaan Hive:

1. Tambahkan dependency hive_flutter pada pubspec.yaml.
2. Inisialisasi dengan Hive.initFlutter() sebelum aplikasi dijalankan (di main.dart).
3. Buka box dengan Hive.openBox('pengeluaran_box').
4. Simpan data berupa Map ke dalam box, lalu baca, ubah, dan hapus berdasarkan key.

Tabel 3. Operasi CRUD pada Hive

Operasi	Method	Fungsi Hive
Create	tambah()	box.add({...}), key dibuat otomatis
Read	semua()	box.keys, lalu box.get(key) untuk tiap key
Update	ubah()	box.put(key, {...})
Delete	hapus()	box.delete(key)

Implementasinya ada di lib/data/hive_repo.dart. Karena HiveRepo memakai antarmuka PengeluaranRepo yang sama dengan SqliteRepo, tampilan dan fitur pada tab Hive identik dengan tab SQLite. Yang berbeda hanya mesin penyimpanannya, sehingga keduanya dapat dibandingkan secara adil.

6. Ringkasan dan Dokumentasi Hasil Uji Coba

6.1 Perbandingan SQLite dan Hive

Tabel 4. Perbandingan SQLite (sqflite) dan Hive

Aspek	SQLite (sqflite)	Hive
Jenis	Relasional (SQL)	NoSQL key-value
Skema	Wajib (tabel dan kolom)	Fleksibel, tanpa skema
Query kompleks	Kuat (WHERE, ORDER BY, JOIN)	Terbatas
Kemudahan	Perlu menulis SQL	API sederhana
Relasi antar data	Didukung	Tidak didukung langsung
Cocok untuk	Data terstruktur dan berelasi	Data sederhana, cache, pengaturan

6.2 Uji Fungsional CRUD

Skenario berikut berlaku sama pada tab SQLite dan tab Hive karena keduanya memakai antarmuka CRUD yang sama.

Tabel 5. Skenario uji fungsional dan hasil yang diharapkan

No	Skenario	Hasil yang diharapkan
1	Aplikasi dibuka pertama kali	Tujuh data contoh tampil di daftar.
2	Tambah pengeluaran (Create)	Data baru muncul di daftar dan total bertambah.
3	Lihat daftar (Read)	Data tampil dari yang terbaru, disertai total dan rincian per kategori.
4	Ubah pengeluaran (Update)	Data berubah di daftar sesuai isian baru.
5	Hapus pengeluaran (Delete)	Data hilang dari daftar setelah digeser ke kiri.
6	Urungkan hapus	Data kembali muncul di daftar.
7	Tutup dan buka kembali aplikasi	Data tetap tersimpan.
8	Kirim form dengan isian kosong	Muncul pesan kesalahan dan data tidak tersimpan.

6.3 Uji Kecepatan

Uji kecepatan dijalankan melalui tab Uji Coba pada aplikasi. Pengujian melakukan operasi tambah, baca, ubah, dan hapus sebanyak 100 data pada SQLite dan Hive, masing-masing di database terpisah sehingga data aplikasi tidak terganggu. Waktu diukur dengan Stopwatch dalam satuan milidetik. Semakin kecil angkanya, semakin cepat operasinya, dan hasil dapat berbeda pada setiap perangkat.

Perangkat uji: .....

Tabel 6. Hasil uji kecepatan (100 data)

Operasi	SQLite (ms)	Hive (ms)
Tambah 100 data		
Baca semua data		
Ubah 100 data		
Hapus 100 data		

6.4 Kesimpulan

SQLite melalui package sqflite dapat dipakai untuk membuat database, skema tabel, dan seluruh operasi CRUD pada aplikasi Pengeluaran Harian. Teknologi ini cocok untuk data terstruktur yang membutuhkan query dan relasi antar tabel, tetapi perlu merancang skema dan menulis perintah SQL. Hive sebagai alternatif juga dapat menjalankan CRUD yang sama dengan lebih praktis dan ringan karena tidak memerlukan skema maupun SQL, namun kemampuan query dan relasinya terbatas. Pemilihan teknologi sebaiknya disesuaikan dengan

kebutuhan: SQLite untuk data yang kompleks dan berelasi, Hive untuk data sederhana yang menuntut kemudahan penggunaan.

7. Struktur Kode dan Cara Menjalankan

- lib/main.dart: inisialisasi Hive dan aplikasi
- lib/models/pengeluaran.dart: model data, kategori, format Rupiah dan tanggal
- lib/data/pengeluaran_repo.dart: antarmuka CRUD
- lib/data/sqlite_repo.dart: implementasi SQLite
- lib/data/hive_repo.dart: implementasi Hive
- lib/data/benchmark.dart: uji kecepatan CRUD
- lib/pages/: halaman utama, daftar pengeluaran, dan uji coba

Cara menjalankan: buka folder project, jalankan flutter pub get, lalu flutter run pada emulator atau perangkat Android.